UNIT 4 · TOPIC 4.1

Device Vulnerabilities and Attacks

Assignment

Instructions: 18 practice questions + 3 coding assignment(s), 30 points total. Show your work for short-answer items. Coding assignments may be completed here or in the interactive auto-graded environment on the course site (same problem, same points).

Name: **Date:** **Class:**

Practice Questions

4.1.a · Malware types (viruses, worms, trojans, ransomware)

1. A malware sample spreads from device to device over the network with no user interaction and no host file. Which category best fits? (1 pt)

- ○ A. Virus
- ○ B. Worm
- ○ C. Trojan
- ○ D. Ransomware

2. A user installs a free video converter that secretly opens a remote-control backdoor. Which malware type is this? (1 pt)

- ○ A. Virus
- ○ B. Worm
- ○ C. Ransomware
- ○ D. Trojan

3. Select ALL statements that are TRUE about ransomware. (2 pts)

- □ A. It encrypts victim files and demands payment for the decryption key
- □ B. It self-replicates across the network without user action
- □ C. Offline backups directly neutralize its leverage
- □ D. It always permanently destroys the encrypted files

(Select ALL that apply.)

4. Which property MOST distinguishes a virus from a worm? (1 pt)

- ○ A. Viruses damage files, while worms only steal data
- ○ B. Viruses require a host file and user action to spread; worms spread autonomously
- ○ C. Viruses arrive by email, while worms arrive by USB drive
- ○ D. Viruses can be removed by antivirus, while worms cannot

5. Name the term for the initial path by which malware enters a device (e.g., a phishing email or infected USB drive). (1 pt)

6. An analyst sees that a device's CPU is near 100% and it is sending data to hundreds of IP addresses, yet no user application is open. Which malware type does this MOST suggest? (1 pt)

- ○ A. Ransomware
- ○ B. Worm
- ○ C. Virus
- ○ D. Trojan

4.1.b · Device tampering and physical compromise

7. A security analyst discovers a small adapter connected between a desktop computer's USB port and the keyboard cable. No software alerts have been triggered. What type of physical attack is this? (1 pt)

- ○ A. USB Rubber Ducky
- ○ B. Cold boot attack
- ○ C. Hardware keylogger
- ○ D. BIOS malware

8. Which of the following mitigations specifically address threats arising from physical theft of a laptop hard drive? (2 pts)

- □ A. Full Disk Encryption (FDE)
- □ B. Strong operating system logon passwords
- □ C. Trusted Platform Module (TPM) integration
- □ D. Software-based firewalls

(Select ALL that apply.)

9. Name the keystroke injection tool disguised as a standard USB flash drive that executes pre-programmed commands by mimicking a keyboard. (1 pt)

10. An attacker sprays compressed air (upside down) on RAM chips immediately after a workstation is powered off, then quickly moves the modules to another machine to read their contents. What is this attack called? (1 pt)

- ○ A. Chassis intrusion
- ○ B. Hardware keylogger
- ○ C. USB Rubber Ducky
- ○ D. Cold boot attack

11. A server room has a keypad lock on its door, but an attacker who is already an employee with legitimate badge access removes a hard drive and reads it on a personal laptop. Which control would have stopped this? (1 pt)

- ○ A. A stronger door lock
- ○ B. A longer OS login password
- ○ C. A security camera in the lobby
- ○ D. Full Disk Encryption (FDE)

12. Name the term for a sticker or label on a device's case or port that visibly tears or changes color if removed, letting staff confirm at a glance that the device has not been opened. (1 pt)

4.1.c · IoT device vulnerabilities

13. A smart camera is still logging in with the username/password it shipped with from the factory. What is this vulnerability called? (1 pt)

- ○ A. Outdated firmware
- ○ B. Unauthenticated API
- ○ C. Shadow IoT
- ○ D. Default credentials

14. Why is the 2016 Mirai botnet a significant example in IoT security? (1 pt)

- ○ A. It was the first malware to infect IoT devices
- ○ B. It spread through phishing emails sent to device owners
- ○ C. It encrypted camera footage and demanded ransom from owners
- ○ D. It hijacked IoT devices via default Telnet credentials and used them to launch record-setting DDoS attacks

15. Select ALL statements that are TRUE about IoT device risk. (2 pts)

- □ A. A device that isn't directly internet-facing can still be reached by other devices already on the same local network
- □ B. Manufacturers always issue timely firmware updates for the life of the product
- □ C. UPnP can automatically open a port on a router, exposing a device without the owner realizing it
- □ D. Attackers often want IoT devices as botnet nodes or network pivot points, not for the device's own data

(Select ALL that apply.)

16. Which property MOST distinguishes an EOL (end-of-life) IoT device's risk from a device with an exposed management port? (1 pt)

- ○ A. EOL devices can be fixed by closing their management port
- ○ B. EOL devices are only risky if they still use default credentials
- ○ C. EOL devices no longer receive firmware updates, so any known vulnerability stays permanently unpatched, regardless of ports or credentials
- ○ D. EOL devices are safe once they are disconnected from the cloud service

17. Name the term for devices connected to a network without IT's knowledge, approval, or security oversight. (1 pt)

18. A device's mobile app sends commands to it over the local Wi-Fi network with no encryption at all. What is the MOST direct consequence? (1 pt)

- ○ A. The device automatically becomes part of a botnet
- ○ B. The device's firmware will stop receiving updates
- ○ C. Anyone else on the same network segment can read or alter the traffic in transit
- ○ D. The manufacturer's cloud API becomes unauthenticated

Coding Assignments (Python 3)

4.1.a · Malware types (viruses, worms, trojans, ransomware)

3 pts · auto-graded on the site

Coding problem 4.1.a: Classify malware by behavioral signature

Skill: Analyze Risk · **Maps to:** 4.1.A, 4.1.B · **Language:** Python 3 (pytest)

Problem

A security analyst receives short incident-report strings that describe observed device behavior. Build the classifier that maps each description to the correct malware category.

Categories:

- `"virus"`: attaches to host files; spreads when files are shared or opened
- `"worm"`: self-propagates across the network without user action; autonomous scanning
- `"trojan"`: disguised as legitimate software; creates backdoor / exfiltrates data
- `"ransomware"`: encrypts files; demands payment for decryption key

A report may match exactly one category. If no category matches, return `"unknown"`.

Function contract

Implement `solve(report: str) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single category label string. Matching is case-insensitive substring detection using the indicator sets defined in `interface.md`.

Input

A single string describing observed behavior on a device or network.

Output

One of: `"virus"`, `"worm"`, `"trojan"`, `"ransomware"`, `"unknown"`.

Examples

```
"Device is scanning all ports on the subnet with no user logged in"         -> "worm"
"All .docx files have been encrypted; a ransom note appeared on the desktop" -> "ransomware"
"A free game installer is sending keystrokes to a remote server"            -> "trojan"
"An email attachment infected the file and spread copies to shared folders" -> "virus"
"Battery usage is higher than usual after a long meeting"                   -> "unknown"
```

Constraints

- Deterministic, offline, standard library only.
- Detection by case-insensitive substring match against indicator sets in `interface.md`.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.1.a. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def solve(report: str) -> str:
    """Classify malware by behavioral signature report.

    Args:
        report: A string containing the incident report description.

    Returns:
        The malware category string: "virus", "worm", "trojan", "ransomware", or "unknown".
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

4.1.b · Device tampering and physical compromise

3 pts · auto-graded on the site

Coding problem 4.1.b: Device tampering and physical compromise

Skill: Analyze Risk · **Maps to:** 4.1.B · **Language:** Python 3 (pytest)

Problem

In a physical security audit, we collect logs from system motherboards, BIOS events, and OS events. You must implement a parser that reads these logs and identifies all compromised devices.

A device is considered physically compromised if it triggers any of the following rules: 1. **Chassis Intrusion:** `event_type == "chassis_intrusion"` and `details` contains `"open"` (case-insensitive). 2. **Unauthorized Boot:** `event_type == "boot"` and `details` contains `"usb"`, `"external"`, or `"live"` (case-insensitive). 3. **USB Rubber Ducky:** A `usb_insert` event with `"keyboard"` or `"hid"` in `details` (case-insensitive) followed by a `command` event on the same device within 3 seconds, where the command `details` contains `"powershell"`, `"cmd.exe"`, or `"download"` (case-insensitive).

Function contract

Implement `solve(logs: list) -> list` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the sorted list of compromised device IDs.

Input

A list of dicts:

```
[
    {
        "timestamp": "2026-06-25T12:00:00Z",
        "device_id": "laptop_01",
        "event_type": "boot",
        "details": "Booted from external USB drive"
    }
]
```

Output

A list of strings, e.g. `["laptop_01"]`, sorted alphabetically.

Examples

```
solve([
    {"timestamp": "2026-06-25T10:00:00Z", "device_id": "pc_01", "event_type": "chassis_intrusion", "details":
```

```
"chassis opened detected"}
]) -> ["pc_01"]

solve([
    {"timestamp": "2026-06-25T10:00:00Z", "device_id": "pc_02", "event_type": "usb_insert", "details": "USB
Keyboard connected"},
    {"timestamp": "2026-06-25T10:00:02Z", "device_id": "pc_02", "event_type": "command", "details": "powershell
-nop -w hidden -c Get-Process"}
]) -> ["pc_02"]
```

Constraints

- Pure function, offline, standard library only.
- Timestamps are ISO-8601 strings.
- Sorted return list with unique elements.

Starter code:

```
"""Starter code for 4.1.b. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def solve(logs: list) -> list:
    """Analyze logs to detect physical tampering and device compromise.

    Args:
        logs: A list of log dictionaries, sorted chronologically.

    Returns:
        A list of compromised device IDs, sorted alphabetically.
    """
    # TODO: implement per prompt.md
    raise NotImplementedError("implement solve() — see prompt.md")
```

4.1.c · IoT device vulnerabilities

3 pts · auto-graded on the site

Coding problem 4.1.c: Audit an IoT device for its dominant vulnerability

Skill: Analyze Risk · **Maps to:** 4.1.A, 4.1.C · **Language:** Python 3 (pytest)

Problem

A security analyst is auditing smart devices on a network. Each device is described by a dictionary of attributes. Build the classifier that inspects a device and returns its single most severe vulnerability category, or `"secure"` if none of the checked issues are present.

A device can technically have more than one issue at once; when it does, report only the most severe one, using this fixed priority order (highest severity first):

1. `"default_credentials"`, the device still uses its factory default password 2. `"outdated_firmware"`, the device's firmware is out of date 3. `"exposed_port"`, the device has an insecure management port open (23 = Telnet, 21 = FTP) 4. `"unauthenticated_api"`, the device's API accepts requests with no authentication 5. `"unencrypted_traffic"`, the device's network traffic is not encrypted 6. `"secure"`, none of the above apply

Function contract

Implement `solve(device: dict) -> str` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return the single verdict label string.

Input

A dictionary with the following keys:

- `default_password` (bool), True if the factory default password is still in use
- `firmware_outdated` (bool), True if the firmware is out of date
- `open_ports` (list[int]), list of open network ports on the device
- `unauthenticated_api` (bool), True if the device's API requires no authentication
- `encrypted_traffic` (bool), True if the device's traffic is encrypted (note: False means traffic is UNencrypted, which is the risk condition)

An insecure management port is any of 23 (Telnet) or 21 (FTP) appearing in `open_ports`.

Output

One of: `"default_credentials"`, `"outdated_firmware"`, `"exposed_port"`, `"unauthenticated_api"`, `"unencrypted_traffic"`, `"secure"`.

Examples

```
{"default_password": True, "firmware_outdated": True, "open_ports": [23], "unauthenticated_api": True,
"encrypted_traffic": False}  ->  "default_credentials"
{"default_password": False, "firmware_outdated": True, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "outdated_firmware"
{"default_password": False, "firmware_outdated": False, "open_ports": [23, 443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "exposed_port"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": True,
"encrypted_traffic": True}  ->  "unauthenticated_api"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": False}  ->  "unencrypted_traffic"
{"default_password": False, "firmware_outdated": False, "open_ports": [443], "unauthenticated_api": False,
"encrypted_traffic": True}  ->  "secure"
```

Constraints

- Deterministic, offline, standard library only.
- Priority order is fixed as listed above and must be applied exactly.
- Only ports 23 and 21 count as insecure/exposed management ports.
- Exactly one label returned (string, not list).
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Starter code:

```
"""Starter code for 4.1.c. Student completes the TODO.
This file intentionally FAILS the tests until implemented."""


def solve(device: dict) -> str:
    """Audit an IoT device dict and return its single dominant vulnerability verdict.
```

```
Args:
    device: dict with keys default_password (bool), firmware_outdated (bool),
        open_ports (list[int]), unauthenticated_api (bool), encrypted_traffic (bool).
        Missing keys default to the "safe" value for that key.

Returns:
    One of: "default_credentials", "outdated_firmware", "exposed_port",
    "unauthenticated_api", "unencrypted_traffic", "secure".
"""
# TODO: implement per prompt.md
raise NotImplementedError("implement solve() — see prompt.md")
```